Na poprzednich zajęciach:

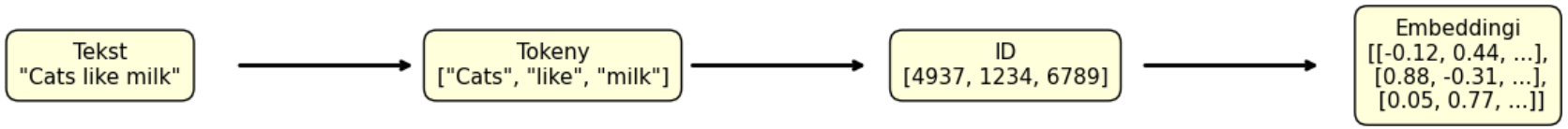
- **tokenizacja** – podział tekstu na słowa / tokeny,
- **spaCy Matcher** – znajdowanie wzorców na podstawie części mowy i cech morfologicznych.

Wektory słów

Tokenizacja a wektory słów

Tokenizacja -> Podziel tekst na kawałki (ID ze słownika)

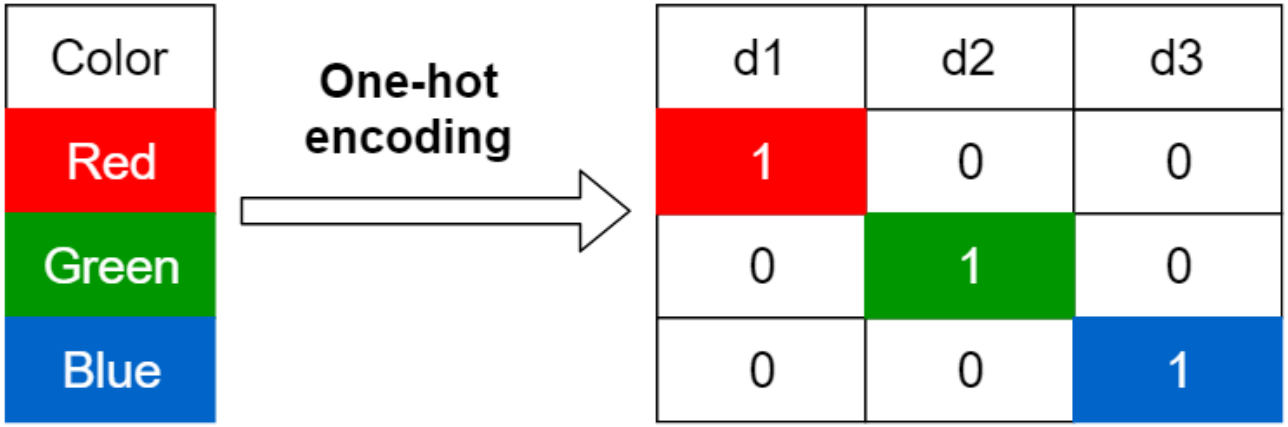
Embedding -> Przedstaw każde słowo w postaci liczb, aby algorytm mógł z nim dalej pracować



Komputer nie „rozumie” tekstu, tylko operuje na liczbach. Zatem każde słowo (np. „król”, „kot”, „miłość”) trzeba jakoś zamienić na liczby, żeby można było nim manipulować w modelu.

Najprostszy pomysł — **one-hot encoding**

```
słownik = [kot, król, królowa]
"król" → [0, 1, 0]
```



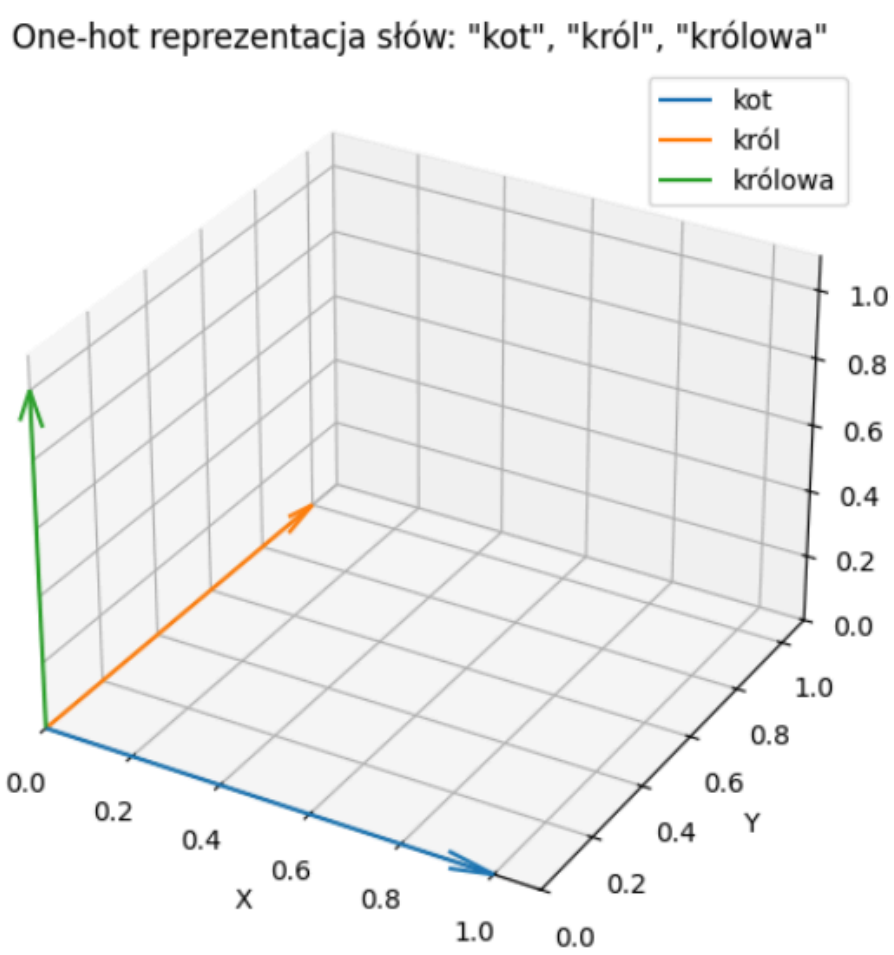
Dlaczego one-hot encoding jest problematyczny?

❌ Bardzo duży wymiar - Jeśli mamy słownik np. 100 000 słów, każdy wektor ma długość 100 000 — to ogromne marnotrawstwo pamięci.

❌ Brak informacji o znaczeniu - Wszystkie wektory są od siebie tak samo odległe — np. “kot” i “pies” są tak samo różne jak “kot” i “samolot”. Nie ma żadnego pojęcia o podobieństwie semantycznym (znaczeniowym).

❌ Brak możliwości uogólniania - Model nie może „zrozumieć”, że pewne słowa mają podobne znaczenie — każde słowo jest całkowicie niezależne.

```
słownik = [kot, król, królowa]
"kot" → [1, 0, 0]
"król" → [0, 1, 0]
"królowa" → [0, 0, 1]
```



Widzimy, że każdy wektor idzie w swoją stronę, każdy jest tak samo odległy od siebie (90 stopni). Żadne dwa nie są do siebie bliżej ani dalej — odległość między każdym z nich jest taka sama.

W naszym przykładzie: kot, król i królowa są tak samo podobni i różni = nie ma wartości semantycznej.

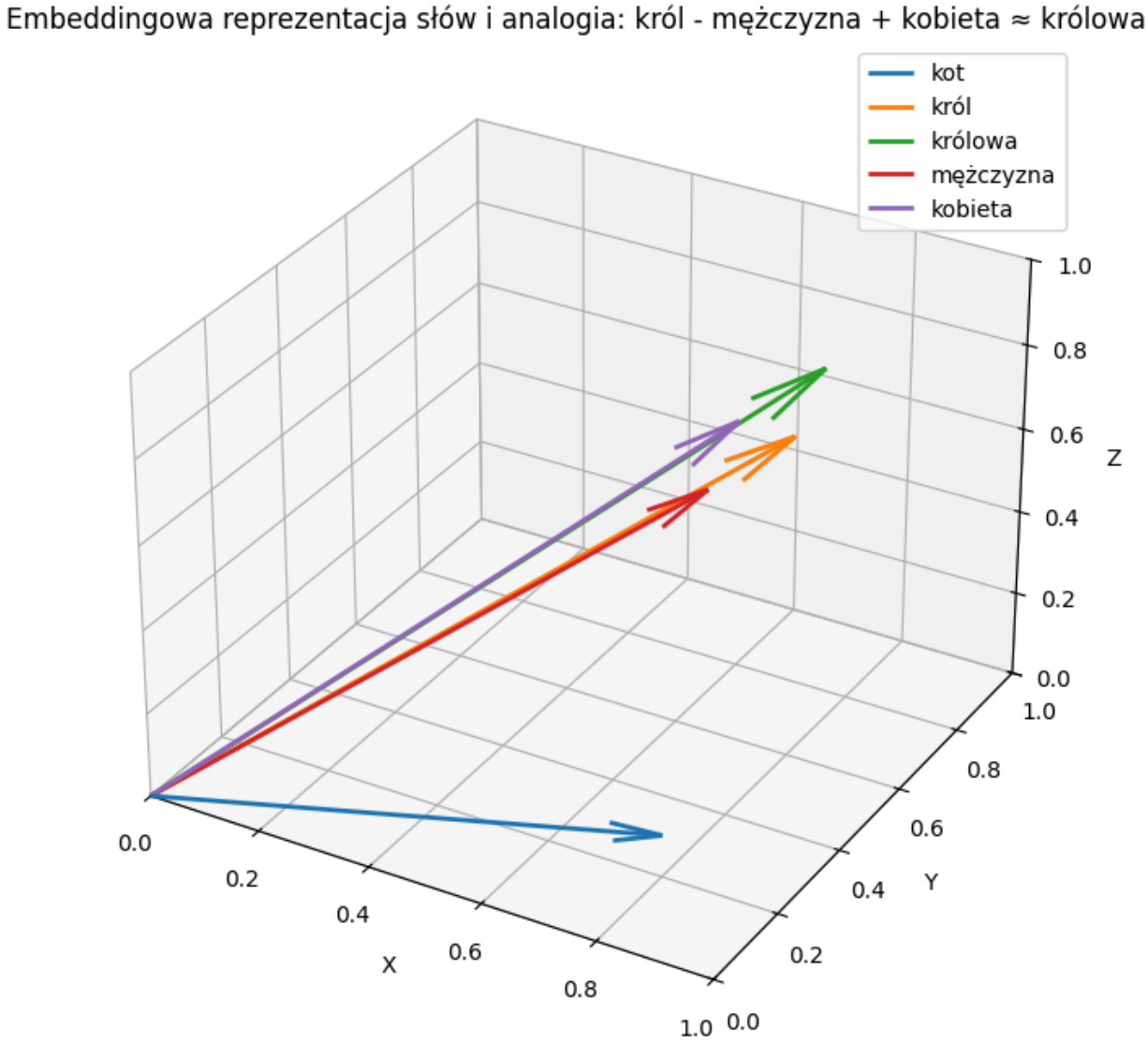
Embedingi

Embeddingi rozwiązują ten problem. W embeddingach uczymy model, żeby słowa, które występują w podobnych kontekstach, miały **podobne wektory**.

Dwa wektory są podobne, kiedy wskazują w podobnym kierunku w przestrzeni wielowymiarowej. Czyli: jeśli narysujesz te wektory jako strzałki zaczynające się w tym samym punkcie (np. w punkcie (0,0,0)) to mają **mały kąt między sobą**.

Przykład:

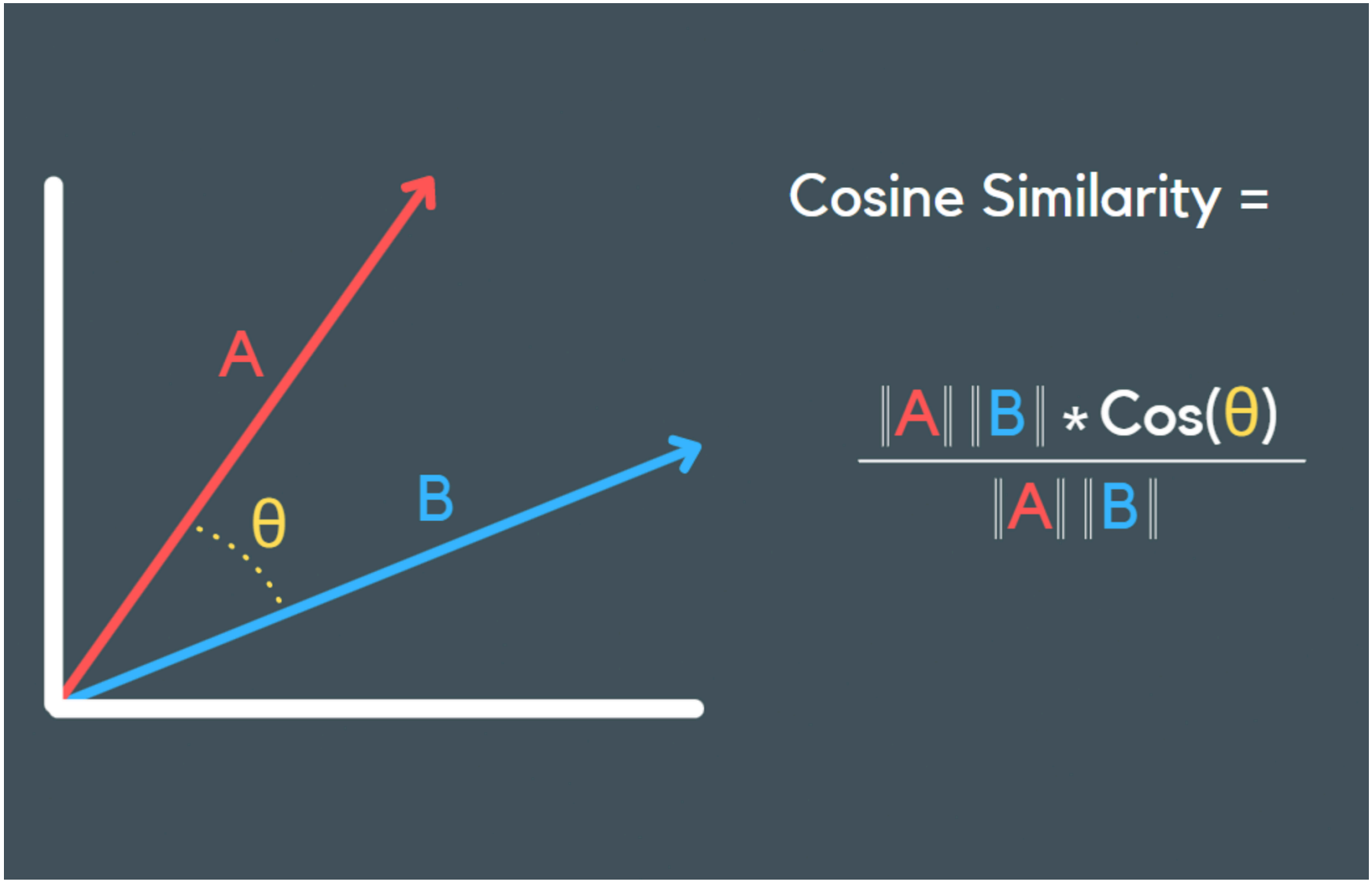
```
"king" → [0.25, 0.88, -0.12, 0.40, ...]
"queen" → [0.22, 0.90, -0.10, 0.45, ...]
"cat" → [-0.70, 0.11, 0.45, 0.20, ...]
```




```
król – mężczyzna + kobieta ≈ królowa
```

Teraz punkty w przestrzeni układają się tak, że słowa o podobnym znaczeniu są blisko siebie. To jest właśnie **semantyczne podobieństwo** — podobieństwo w sensie znaczenia, a nie liter czy indeksów w tabeli.

Cosinusowe podobieństwo — jak mierzymy „bliskość znaczeń”



Cosinusowe podobieństwo to sposób na sprawdzenie, czy dwa wektory są „w tym samym kierunku”:

- 1 → bardzo podobne (np. „kot” i „pies”)

- 0 ⟶ brak podobieństwa (np. „kot” i „okno”)
- -1 ⟶ całkowite przeciwieństwo

Ważne: Patrzymy nie na długość wektorów, tylko na kąt między nimi. Dwa wektory, które „patrzą” w tym samym kierunku, oznaczają słowa o podobnym znaczeniu.

Skąd się biorą te wektory?

Z treningu modelu językowego (np. Word2Vec, GloVe, FastText, BERT). Model uczy się na dużych korpusach tekstu, obserwując konteksty słów.

Efekt:

„king” i „queen” często występują w podobnych zdaniach (np. royal palace, throne, reign) ⟶ mają podobne wektory.

„dog” i „cat” też ⟶ inna grupa semantyczna.

Trochę praktyki

Zaczniemy od zapoznania się z biblioteką **gensim** Jej nazwa pochodzi od "GENerate SIMilar" — bo jej głównym celem jest znajdowanie podobieństwa między dokumentami lub słowami.

Użyjemy klasy **KeyedVectors** - to strukturą danych w gensim, która przechowuje wektory słów (embeddingi) wytrenowane dowolnym modelem. Czyli po prostu taki kontener do pracy z wektorami - niezależnie od tego, jak one zostały wytrenowane.

```
!pip install gensim
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.13/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.13/dist-packages (from gensim) (2.1.3)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.13/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.13/dist-packages (from gensim) (8.0.2)
Requirement already satisfied: wrapt in /usr/local/lib/python3.13/dist-packages (from smart_open>=1.8.1->gensim) (2.5.0)
```

```
import gensim
from gensim.models import KeyedVectors
```

weźmiemy sobie polskie pre-trenowane wektory: http://dsmodels.nlp.ipipan.waw.pl pobieramy plik:

```
!wget http://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz --no-check-certificate
```

```
--2026-10-15 10:53:39--  http://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz
Resolving dsmodels.nlp.ipipan.waw.pl (dsmodels.nlp.ipipan.waw.pl)... 213.135.36.94
```

```
Connecting to dsmodels.nlp.ipipan.waw.pl (dsmodels.nlp.ipipan.waw.pl)|213.135.36.94|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz [following]
--2026-10-15 10:53:40--  https://dsmodels.nlp.ipipan.waw.pl/dsmodels/nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz
Connecting to dsmodels.nlp.ipipan.waw.pl (dsmodels.nlp.ipipan.waw.pl)|213.135.36.94|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2152612134 (2.0G) [application/octet-stream]
Saving to: ‘nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz’

nkjp+wiki-forms-res 100%[===================>]   2.00G  21.0MB/s    in 1m 44s

2026-10-15 10:55:24 (19.7 MB/s) - ‘nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz’ saved [2152612134/2152612134]
```

Rozpakowujemy dane

```
!gzip -d "nkjp+wiki-forms-restricted-300-skipg-ns.txt.gz"
```

```
gzip: nkjp+wiki-forms-restricted-300-skipg-ns.txt already exists; do you wish to overwrite (y or n)? n
        not overwritten
```

Badamy format pliku

```
!head -n 100 nkjp+wiki-forms-restricted-300-skipg-ns.txt
#Pierwsza linia - lość słów (1982479), rozmiar wektora(300)
#Każda kolejna linia - słowo, liczby reperezentujące wektor
```

```
Polsce -0.101274 -0.141963 -0.122373 0.077275 -0.070007 -0.145128 -0.087445 -0.002497 -0.096752 0.377047 0.352206 0.072971 -0.166066 -0.149228 -0.01386
Www -0.169431 -0.745244 -0.178188 -0.024577 0.230027 -0.087595 -0.134648 0.265898 0.365070 -0.645357 0.018781 0.603436 -0.088387 -0.091858 -0.093921 0
ta -0.244184 -0.063868 -0.287503 0.362653 -0.122516 -0.218331 0.032908 -0.144951 0.020826 0.083380 0.027182 0.154032 0.345063 -0.510692 -0.205026 -0.03
kto 0.118961 -0.258968 -0.112432 0.067625 0.087058 0.013574 0.143056 -0.078806 -0.174179 0.013723 0.320065 0.036876 0.210094 -0.166539 0.132591 -0.2218
nic -0.321820 0.124438 0.119892 -0.114817 -0.143212 0.324809 -0.202791 0.063554 -0.136038 -0.105519 -0.020312 0.180893 0.034437 -0.119624 -0.040387 -0
innych -0.165053 -0.013792 -0.039466 0.426864 0.069514 -0.124087 -0.052136 0.113360 0.182566 0.723688 0.019989 -0.001580 0.174827 -0.038742 0.211826 -0
raz -0.320460 0.181834 0.696714 0.040844 -0.466045 0.235893 -0.198609 -0.163290 -0.216640 0.177298 -0.134643 -0.010222 0.086820 -0.558299 -0.072314 -0
którym 0.320436 0.117971 0.011519 0.023817 0.014940 -0.025448 0.206506 -0.684616 0.093412 0.355907 0.097588 -0.079949 0.118874 -0.643311 0.048982 -0.24
dlatego -0.108311 0.002148 -0.027184 0.039365 -0.163655 0.045208 0.042392 -0.084249 0.021344 -0.025603 0.109288 -0.155198 0.219184 -0.360182 0.077606
np -0.160001 0.034499 -0.162794 0.327848 0.113743 -0.143355 -0.083623 -0.022349 0.004688 0.336429 -0.044344 0.024681 0.279189 -0.099055 -0.007851 -0.16
sposób -0.201393 -0.014430 -0.092039 -0.058065 0.061603 -0.214637 0.005381 -0.171318 -0.003409 0.330691 -0.221737 -0.154607 0.218749 -0.257072 0.078849
ul -0.098716 -0.370421 0.266958 0.478433 0.309188 -0.204205 -0.077770 0.099785 0.467303 0.157737 0.077627 -0.046563 0.094974 0.164292 -0.093150 0.05083
jeden -0.154086 0.292675 0.301450 0.181307 -0.093925 0.344033 -0.347356 -0.269718 0.059804 0.273640 -0.049188 0.176493 0.593115 -0.243326 0.262040 -0.3
onet 0.252580 -0.631651 0.285944 0.450814 -0.186678 -0.166855 -0.303163 0.299909 -0.500637 -0.361839 -0.117994 0.187422 -0.353624 0.377035 0.677921 -0
wszystkich -0.535150 -0.246920 0.192077 0.078758 0.211076 -0.007186 0.055192 0.097090 0.107758 0.428338 -0.190776 0.002887 0.098937 -0.265153 -0.133377
ludzi -0.146394 -0.051471 0.284342 0.108576 -0.037665 0.286483 -0.233572 0.090869 0.179447 0.349003 -0.168273 -0.326376 -0.026598 -0.451043 0.378625 -0
coś -0.132174 0.221940 -0.113811 -0.215245 0.014817 0.151224 -0.117561 0.027730 -0.353648 0.082343 0.097456 0.061694 0.034049 -0.170295 0.023023 -0.086
wtedy -0.103848 0.113345 0.081170 0.003586 -0.077880 -0.144664 0.117029 -0.198689 -0.064029 0.472859 -0.026057 -0.122667 0.108575 -0.492598 0.006071 0
Polski 0.215480 0.156795 0.497221 0.030640 -0.099362 0.360297 -0.249186 -0.212894 0.091294 0.299240 -0.104470 -0.118039 0.225425 0.436383 0.208027 -0.2
sam 0.046605 0.091423 -0.048945 -0.042463 -0.269801 -0.226313 -0.129501 -0.027261 -0.079784 0.396620 -0.020117 -0.009558 0.394802 -0.397545 0.346472 -0
bardziej -0.317128 0.122344 -0.328702 0.182841 0.032999 0.094599 -0.025545 0.184788 0.175031 0.043342 0.191098 -0.209697 0.430397 -0.317807 0.040523 -0
której 0.066885 -0.160596 0.138113 0.135752 0.090126 -0.242285 -0.076305 -0.497955 0.311426 0.406131 0.017828 -0.114246 0.249295 -0.474698 -0.166636 -0
dobrze 0.113636 0.063054 -0.014993 0.162755 -0.265659 -0.069123 -0.082335 0.058954 -0.066158 0.207598 -0.229820 0.091295 0.134736 -0.139341 0.163519 -0
mogą -0.114524 0.349660 -0.057291 0.243569 0.189238 0.144260 -0.160020 -0.214434 0.241508 0.275314 -0.526742 -0.038836 0.682631 -0.197982 -0.410072 -0
tys -0.318129 -0.175270 0.298398 0.183666 -0.034310 0.092605 0.153846 0.064762 0.204168 0.531119 0.162973 -0.345094 -0.049763 -0.475467 -0.062835 0.003
wszystkie -0.384140 0.064859 0.235217 0.265926 -0.018668 -0.043197 -0.020812 -0.197822 -0.192180 0.250402 -0.284625 0.033763 0.269073 -0.610661 -0.1762
pozdrawiam 0.041227 -0.506711 -0.464303 -0.258203 0.085803 -0.095673 -0.100222 0.145153 0.113782 -0.183297 -0.097734 0.261279 -0.308386 0.173315 -0.044
czasie -0.060830 0.231893 -0.105014 0.282289 -0.235925 0.079309 0.197749 0.129203 0.334069 0.387241 -0.092672 -0.132635 0.224742 -0.487458 0.010481 -0
zawsze -0.061509 -0.208795 0.002352 0.028405 -0.161910 -0.031843 -0.245843 0.051215 0.022118 0.203113 -0.294965 -0.127762 0.042376 -0.301025 -0.138677
pierwszy -0.266575 0.384896 0.660251 0.305221 -0.381303 0.212457 -0.357796 -0.329724 -0.171845 0.295718 -0.111321 -0.036993 0.249035 -0.617400 0.056781
jestem -0.028048 0.179036 -0.072213 -0.056222 0.007244 0.189300 0.034070 0.109402 -0.237497 -0.065510 -0.017094 0.150471 0.024600 0.114804 -0.078838 -0
czas -0.018453 0.137894 0.065990 0.001308 -0.044101 -0.204609 0.361818 -0.072674 0.096675 0.172855 -0.257486 -0.105790 0.253550 0.106275 -0.326278 -0.2
dnia -0.081396 0.056075 0.253912 0.308968 -0.199097 -0.115671 0.008291 0.250811 0.487077 0.468913 -0.318629 -0.008835 -0.221629 -0.316106 -0.011876 0.2
została -0.628995 -0.172253 -0.363241 0.600747 -0.374822 -0.125494 -0.159600 -0.193493 -0.106565 0.311173 -0.134814 -0.046976 0.162803 -0.229352 -0.329
dziś -0.015517 -0.001961 -0.005589 0.377691 0.294157 -0.192328 0.066951 -0.038222 0.058811 0.372265 0.037670 0.227869 0.322231 0.060120 -0.007307 0.201
później 0.000400 0.341058 0.136597 0.256254 -0.182658 -0.218749 -0.318174 -0.047242 -0.102294 0.438479 -0.237589 -0.253496 0.127474 -0.356090 -0.14389
```

(to może chwilę zająć)

```
#Ładowanie wektorów z pliku do naszego KeyedVectors
wv = KeyedVectors.load_word2vec_format('nkjp+wiki-forms-restricted-300-skipg-ns.txt')
```

```
wv.key_to_index.keys()
wv['dom']
```

```
array([-0.39294 , -0.156333,  0.017816, -0.208752,  0.442839, -0.389071,
       -0.22354 ,  0.119761,  0.201861,  0.136966,  0.005829, -0.036992,
       -0.075693, -0.424958,  0.031507, -0.214998, -0.11862 , -0.059799,
       -0.150971, -0.176567,  0.444136,  0.18925 ,  0.241265, -0.248733,
```

```
       -0.176206,  0.579075,  0.031083,  0.253453,  0.101754,  0.144021,
       -0.169662, -0.005122,  0.056225,  0.078038, -0.557848, -0.365922,
       -0.108863,  0.322074, -0.178232,  0.289982, -0.205699,  0.001991,
        0.001029, -0.074944,  0.173145,  0.044346, -0.032673, -0.060124,
        0.152098, -0.293121, -0.280878,  0.377744,  0.402627, -0.737609,
        0.020663, -0.01301 ,  0.134813, -0.052369,  0.046645, -0.024955,
        0.100392, -0.295437, -0.418354,  0.396864,  0.232666,  0.022819,
       -0.018474,  0.385309, -0.084371,  0.269105,  0.149824, -0.161652,
        0.282115,  0.177849, -0.241524, -0.240634, -0.111456,  0.070668,
       -0.216499, -0.139156, -0.576622, -0.136423,  0.403575,  0.155968,
        0.218948, -0.002413,  0.123077,  0.102393,  0.741444,  0.390994,
       -0.110964,  0.134329,  0.265171,  0.251443, -0.221384, -0.344603,
        0.283788,  0.284915,  0.057016,  0.384028, -0.733822,  0.303532,
       -0.583847, -0.02494 , -0.218198,  0.018208,  0.145667,  0.255729,
        0.498478, -0.112536, -0.047852, -0.337887,  0.031235, -0.432209,
        0.208977,  0.012221,  0.19098 , -0.206233, -0.113016,  0.111511,
       -0.059505,  0.027593, -0.100111, -0.230436, -0.078137,  0.278976,
        0.19794 ,  0.210459, -0.155359, -0.112397,  0.26439 , -0.199999,
        0.058368,  0.032107,  0.110054,  0.051876, -0.240713,  0.277577,
        0.289985, -0.200094, -0.101406,  0.131032, -0.059501,  0.134268,
        0.148928, -0.085373,  0.030669,  0.068627, -0.215063, -0.081981,
       -0.052877, -0.291941, -0.458875,  0.050618,  0.214273,  0.184849,
       -0.038208,  0.036236, -0.642125, -0.464941,  0.372085, -0.187011,
       -0.069729,  0.035051,  0.099067,  0.057524,  0.006579,  0.533439,
       -0.198236, -0.344194, -0.067796, -0.063289, -0.069952, -0.041706,
        0.142563,  0.05342 , -0.054871,  0.2577  ,  0.50673 ,  0.208496,
        0.650207, -0.252643,  0.320542, -0.54734 , -0.637634, -0.51038 ,
        0.025401,  0.05803 ,  0.16656 ,  0.257388,  0.305958, -0.219199,
        0.2086  , -0.251887, -0.185647, -0.027838, -0.186104, -0.070295,
       -0.094713,  0.102284, -0.123802,  0.083343,  0.315851, -0.031123,
        0.238093,  0.149717, -0.004316, -0.181932, -0.086956,  0.531083,
        0.112422,  0.231151,  0.242227,  0.012977, -0.095725, -0.283862,
       -0.061226,  0.311026,  0.146489,  0.054716, -0.048658, -0.282848,
       -0.054836, -0.030167, -0.090273, -0.435833, -0.287074, -0.351567,
       -0.002671, -0.020709, -0.276286,  0.281762, -0.114932, -0.243566,
       -0.188988,  0.007132,  0.146958, -0.269132, -0.034678,  0.164219,
       -0.090405,  0.044055, -0.405617,  0.162068,  0.008641,  0.142725,
       -0.065804,  0.136329,  0.247081, -0.144136, -0.007592,  0.113807,
        0.14899 , -0.012305, -0.346247,  0.111213,  0.011855,  0.250259,
        0.068319,  0.100238,  0.010446,  0.127471, -0.033004, -0.079141,
       -0.531317,  0.318433,  0.562701,  0.354442,  0.001789,  0.171425,
        0.010197,  0.043681, -0.031715,  0.505568, -0.453908,  0.050882,
       -0.294158, -0.279005, -0.121981,  0.182517,  0.419662,  0.325643,
       -0.027132, -0.020383,  0.133923, -0.047128,  0.406531, -0.385169,
       -0.148565, -0.109004, -0.062878, -0.236207,  0.008543,  0.044215,
        0.216761,  0.017197, -0.034889,  0.311272,  0.130243, -0.122846],
      dtype=float32)
```

Znajdziemy najbardziej podobne słowo do słowa kobieta.

```
wv.most_similar('kobieta')
```

```
[('mężczyzna', 0.7043285369873047),
 ('dziewczyna', 0.6813511252403259),
 ('ciężarna', 0.612939715385437),
 ('dziewczynka', 0.6094770431518555),
 ('50-letnia', 0.5805984139442444),
 ('młoda', 0.5788260102272034),
 ('40-letnia', 0.5631040930747986),
 ('kobietą', 0.5611501336097717),
 ('dama', 0.5526077747344971),
 ('mężczyzną', 0.5519220232963562)]
```

Znajdź najbardziej podobne słowo do słowa "kobieta", które nie jest podobne do słowa "mężczyzna"

```
wv.most_similar(positive=['kobieta'], negative=['mężczyzna'])
```

```
[('Teresa', 0.3597542345523834),
 ('Irlandka', 0.35389503836631775),
 ('Barbara', 0.33941537141799927),
 ('Helena', 0.33837634325027466),
 ('Eleonora', 0.32829549908638),
 ('mianowana', 0.3252837657928467),
 ('Maryse', 0.32269802689552307),
 ('Emmanuella', 0.32241636514663696),
 ('czarnoskóra', 0.3224010169506073),
 ('Izabela', 0.3222866654396057)]
```

W grupie "pies kot żubr budyń" znajdź słowo, które nie pasuje.

```
wv.doesnt_match(['pies', 'kot', 'żubr', 'budyń'])
```

```
'budyń'
```

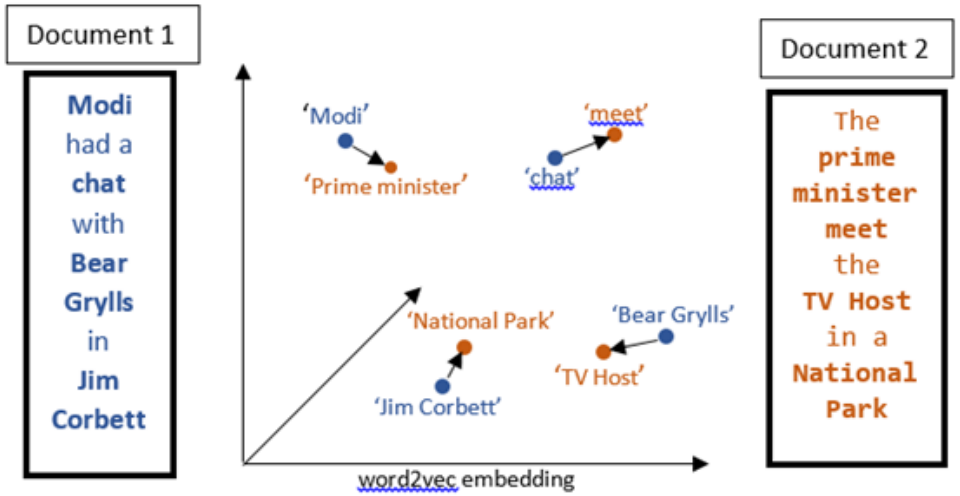
zbadaj podobieństwo między kotem a psem

```
wv.similarity('kot', 'pies')
```

```
np.float32(0.6970217)
```

Teraz zajmiemy się podobieństwami pomiędzy zdaniami

Word mover distance to minimalna suma odległości, jaką trzeba „pokonać”, aby słowa z jednego zdania pokryły znaczeniowo słowa z drugiego zdania.




```
!pip install POT #Po tej instalacji gensim może szybciej i dokładniej liczyć wmdistance
```

```
Collecting POT
  Downloading pot-0.9.7.post1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (44 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.2/44.2 kB 1.8 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.16 in /usr/local/lib/python3.13/dist-packages (from POT) (2.1.3)
Requirement already satisfied: scipy>=1.6 in /usr/local/lib/python3.13/dist-packages (from POT) (1.16.3)
Downloading pot-0.9.7.post1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (32.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 32.9/32.9 MB 19.9 MB/s eta 0:00:00
Installing collected packages: POT
Successfully installed POT-0.9.7.post1
```

```
sentence_1 = 'Byłem dzisiaj w kinie'.lower().split()
sentence_2 = 'Poszłam wczoraj do teatru'.lower().split()
```

```
wv.wmdistance(sentence_1, sentence_2)
```

```
1.176677407878612
```

WMD nie jest ograniczone do 0-1, ale przy normalizacji embeddingów zwykle wartości mieszczą się w 0-4 dla krótkich zdań.

wynik ~1.25 sugeruje, że zdania są umiarkowanie podobne semantycznie.

Dla porównania:

- Dwa identyczne zdania → WMD ≈ 0

- Zupełnie różne zdania → WMD > 2.5-3

Flair

Flair to biblioteka NLP od Zalando Research (Python)

Główna idea:

- Klasyczne embeddingi (Word2Vec, GloVe, FastText) dają stały wektor dla każdego słowa, niezależnie od kontekstu:

  "bank" w *"river bank"* i *"money bank"* → ten sam wektor.

- Flair tworzy kontekstowe embeddingi(wektor słowa zależy od jego zdania / kontekstu)

  więc "*bank*" w "*river bank*" będzie inny niż w "*money bank*".

Flair uczy się tego przez modele LSTM działające w dwóch kierunkach (forward i backward).

```
!pip install flair
```

```
  Created wheel for langdetect: filename=langdetect-1.0.9-py3-none-any.whl size=993335 sha256=76e86eeef12e5602085d30adca8a02670e1ff490bc461a3d1e53a10f1
  Stored in directory: /root/.cache/pip/wheels/eb/87/25/2dddf1c94e1786054e25022ec5530bfed52bad86d882999c48
  Building wheel for pptree (setup.py) ... done
  Created wheel for pptree: filename=pptree-3.1-py3-none-any.whl size=4641 sha256=518355ad04f22ee871fdcfd7fb757567c64445790b291380d2fbdefce2a94cc1
  Stored in directory: /root/.cache/pip/wheels/97/5c/d1/a8347c9c986fc222c12240fafdde88e50a72553e19d7517177
  Building wheel for sqlitedict (setup.py) ... done
  Created wheel for sqlitedict: filename=sqlitedict-2.1.0-py3-none-any.whl size=16958 sha256=9f9297703235fd80df0de66755bb222c89a64a23ce3782d864672f7781
  Stored in directory: /root/.cache/pip/wheels/56/1e/68/fc282426da812281e81368e59ed19f3edc6d9eef4518dbc41c
  Building wheel for docopt (setup.py) ... done
  Created wheel for docopt: filename=docopt-0.6.2-py2.py3-none-any.whl size=13781 sha256=c6b1fb4ab435b3dddffe2d92c48a3dddf71059c9033ffb5b44a442148d0df6
  Stored in directory: /root/.cache/pip/wheels/0b/1d/03/175286677fb5a1341cc3e4755bf8ec0ed08f3329afd67446b0
Successfully built langdetect pptree sqlitedict docopt
Installing collected packages: sqlitedict, pptree, docopt, segtok, langdetect, jsonlines, jmespath, intervaltree, ftfy, deprecated, conllu, huggingface
  Attempting uninstall: huggingface-hub
    Found existing installation: huggingface_hub 1.33.0
    Uninstalling huggingface_hub-1.33.0:
      Successfully uninstalled huggingface_hub-1.33.0
  Attempting uninstall: tokenizers
    Found existing installation: tokenizers 0.23.2
    Uninstalling tokenizers-0.23.2:
      Successfully uninstalled tokenizers-0.23.2
  Attempting uninstall: transformers
    Found existing installation: transformers 5.18.0
    Uninstalling transformers-5.18.0:
      Successfully uninstalled transformers-5.18.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the followir
diffusers 0.40.0 requires huggingface-hub<2.0,>=1.23.0, but you have huggingface-hub 0.36.2 which is incompatible.
gradio 6.29.0 requires huggingface-hub<3.0,>=1.16.0, but you have huggingface-hub 0.36.2 which is incompatible.
Successfully installed bioc-2.1 boto3-1.43.114 botocore-1.43.114 conllu-4.5.3 deprecated-3.0.0 docopt-0.6.2 flair-0.15.1 ftfy-6.3.1 huggingface-hub-0.3
```

```
from flair.embeddings import FlairEmbeddings, StackedEmbeddings
from flair.data import Sentence
```

```
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernels will not work
```

```
embeddings = FlairEmbeddings('news-forward')

#tworzymy objekty Sentence
s1 = Sentence("How does it feel to be a scribe?")
s2 = Sentence("There is no such thing as simply good or bad")
sentences = [s1, s2]


# Obliczamy embeddingi(zanurzamy słowa)
embeddings.embed(sentences)

for sentence in sentences: #Zanurzone zdania nadal są obiektem Sentence
  print(sentence)
```

```
2026-10-15 11:15:00,583 https://flair.informatik.hu-berlin.de/resources/embeddings/flair/news-forward-0.4.1.pt not found in cache, downloading to /tmp/t
100%|██████████| 69.7M/69.7M [00:13<00:00, 5.44MB/s]2026-10-15 11:15:14,432 copying /tmp/tmp7yoqkq14 to cache at /root/.flair/embeddings/news-forward-0.

2026-10-15 11:15:14,501 removing temp file /tmp/tmp7yoqkq14
Sentence[9]: "How does it feel to be a scribe?"
Sentence[10]: "There is no such thing as simply good or bad"
```

```
#dostajemy embeddingi poszczególnych słów
for sentence in sentences:
  for token in sentence:
    print(token.embedding)
#embeddingi są tensorami
```

```
tensor([-0.0011, -0.0021,  0.0038,  ..., -0.0004,  0.0099,  0.0547])
tensor([-0.0016, -0.0260,  0.0388,  ...,  0.0031, -0.0070,  0.0025])
tensor([0.0174, 0.0181, 0.0148,  ..., 0.0051, 0.0051, 0.0130])
tensor([ 0.0001,  0.0128,  0.0476,  ..., -0.0084, -0.0023,  0.0106])
tensor([-2.1672e-03,  8.3753e-05,  2.5102e-02,  ..., -2.8347e-04,
        -1.2090e-02,  2.9513e-02])
tensor([-0.0032,  0.0020,  0.0280,  ...,  0.0004, -0.0017, -0.0106])
tensor([ 5.3283e-03,  3.8049e-05, -2.2042e-02,  ..., -4.1646e-04,
         1.9279e-02, -1.2351e-02])
tensor([-1.0594e-03,  4.8670e-05,  2.1462e-02,  ..., -1.1294e-05,
        -4.3308e-03,  3.1280e-03])
tensor([ 8.7317e-04,  3.7862e-06, -4.6351e-04,  ..., -1.4222e-04,
         5.4708e-03,  3.0888e-03])
tensor([-0.0012, -0.0002, -0.1499,  ...,  0.0024,  0.0017,  0.1648])
tensor([ 0.0078, -0.0047,  0.0564,  ..., -0.0012,  0.0325,  0.0035])
tensor([ 5.8074e-02, -7.9190e-05,  9.8895e-02,  ..., -2.7518e-03,
         8.8326e-04,  2.6957e-02])
tensor([-0.0004, -0.0036,  0.0781,  ..., -0.0004, -0.0344, -0.0005])
tensor([ 0.0014,  0.0033, -0.0118,  ..., -0.0005, -0.0164,  0.0293])
tensor([ 1.2536e-02, -1.0569e-03, -1.1876e-02,  ..., -2.9878e-05,
         4.0988e-03,  2.8744e-02])
tensor([-0.0053, -0.0011,  0.0058,  ..., -0.0018,  0.0212,  0.0082])
tensor([ 2.0060e-04,  9.6346e-04, -2.2168e-02,  ...,  6.8233e-05,
        -1.3982e-01,  3.5051e-03])
tensor([-9.0032e-05, -1.7683e-03,  1.0521e-02,  ..., -1.9229e-03,
        -1.2467e-05,  3.2030e-02])
tensor([-0.0006,  0.0004, -0.0019,  ..., -0.0005,  0.0061, -0.0073])
```

```
import numpy as np
```

```
for sentence in sentences:
  for token in sentence:
    print(np.array(token.embedding))
```

```
[-0.00108573 -0.00213397  0.00377833 ... -0.00035715  0.0099433
  0.05467612]
[-0.00163258 -0.02604993  0.03880594 ...  0.00308893 -0.00699765
  0.00248321]
[0.01740974 0.01807804 0.0147752  ... 0.00507008 0.00508688 0.01302868]
[ 0.00013419  0.01282798  0.04760626 ... -0.00837356 -0.00231261
  0.01057426]
[-2.1671702e-03  8.3752588e-05  2.5102206e-02 ... -2.8347093e-04
 -1.2090425e-02  2.9513163e-02]
[-0.00319913  0.00195263  0.02804686 ...  0.00040722 -0.00174229
 -0.01062463]
[ 5.32832043e-03  3.80494457e-05 -2.20420863e-02 ... -4.16457391e-04
  1.92785766e-02 -1.23509625e-02]
[-1.0594028e-03  4.8669703e-05  2.1461956e-02 ... -1.1294257e-05
 -4.3307757e-03  3.1279572e-03]
[ 8.7317190e-04  3.7861769e-06 -4.6351264e-04 ... -1.4222013e-04
  5.4707765e-03  3.0888412e-03]
[-0.00118427 -0.00018427 -0.14988413 ...  0.00244023  0.00166776
  0.16482687]
[ 0.00780449 -0.00474085  0.056434   ... -0.00120921  0.03250852
  0.00350805]
[ 5.8073573e-02 -7.9189987e-05  9.8894574e-02 ... -2.7517818e-03
  8.8326377e-04  2.6957348e-02]
[-0.0003961  -0.00361209  0.07808607 ... -0.00039112 -0.03444917
 -0.00053953]
[ 0.00144947  0.00326684 -0.01180778 ... -0.00050383 -0.01641377
  0.02928468]
[ 1.2535800e-02 -1.0568546e-03 -1.1875577e-02 ... -2.9878282e-05
  4.0988261e-03  2.8744373e-02]
[-0.00529048 -0.00108928  0.00580226 ... -0.00176485  0.02117513
  0.00820003]
[ 2.0059671e-04  9.6345815e-04 -2.2167904e-02 ...  6.8233297e-05
 -1.3981722e-01  3.5051273e-03]
[-9.00320447e-05 -1.76826888e-03  1.05207525e-02 ... -1.92288996e-03
 -1.24673452e-05  3.20298225e-02]
[-0.00058445  0.00037377 -0.00193683 ... -0.00046098  0.00614759
 -0.00726028]
/tmp/ipykernel_3119/932037997.py:3: DeprecationWarning: __array__ implementation doesn't accept a copy keyword, so passing copy=False failed. __array__
  print(np.array(token.embedding))
```

Embeddingi dla zdań

```
from flair.embeddings import DocumentPoolEmbeddings
```

```
sentence_embeddings = DocumentPoolEmbeddings([FlairEmbeddings('news-forward')])
```

```
sentence_embeddings.embed(sentences)
```

```
for sentence in sentences:
  for token in sentence:
    print(token.embedding)
```

```
tensor([-5.4509e-03, -2.1590e-02,  4.5458e-05,  ...,  9.3315e-03,
         5.7862e-05, -4.2554e-02])
tensor([-0.0321,  0.0122, -0.0004,  ...,  0.0342, -0.0003, -0.0929])
tensor([-0.0031,  0.0031,  0.0005,  ...,  0.0012,  0.0009, -0.2052])
tensor([-0.0030,  0.0112,  0.0003,  ...,  0.0085,  0.0004, -0.0097])
tensor([-2.5321e-03,  2.3465e-02,  2.9343e-05,  ..., -3.2340e-03,
         5.4619e-04, -9.1570e-03])
tensor([-1.3641e-03, -2.4845e-03,  7.5957e-05,  ...,  9.7579e-03,
         1.3047e-03, -1.0619e-01])
tensor([ 2.0078e-02, -1.6552e-02, -3.9296e-05,  ...,  4.9687e-03,
        -3.5058e-06,  2.1542e-02])
tensor([-0.0068, -0.0069,  0.0022,  ...,  0.0182,  0.0003, -0.0256])
tensor([-0.0095,  0.0093, -0.0002,  ...,  0.0327, -0.0003, -0.0622])
tensor([ 6.3525e-05, -8.6742e-03,  2.9983e-03,  ...,  1.1318e-03,
        -9.2730e-05, -3.5767e-02])
tensor([-0.0015, -0.0156,  0.0006,  ...,  0.0054,  0.0006, -0.0252])
tensor([-0.0043, -0.0131,  0.0004,  ...,  0.0074,  0.0004, -0.0169])
tensor([-0.0002, -0.0049,  0.0003,  ...,  0.0027,  0.0006, -0.1442])
tensor([-1.6349e-02,  2.2101e-03,  6.7118e-03,  ..., -5.4466e-03,
        -7.6566e-05, -1.2558e-02])
tensor([3.6809e-02, 1.3150e-03, 5.3269e-04,  ..., 7.7587e-04, 3.5749e-05,
        4.5764e-03])
tensor([-0.0034, -0.0059,  0.0002,  ...,  0.0146,  0.0003, -0.0051])
tensor([-0.0043, -0.0211,  0.0008,  ...,  0.0061,  0.0003, -0.0081])
tensor([-0.0124,  0.0077,  0.0005,  ...,  0.0263, -0.0002,  0.0411])
tensor([-0.0148, -0.0056,  0.0006,  ...,  0.0037,  0.0002, -0.0400])
tensor([-0.0011, -0.0063,  0.0013,  ...,  0.0090,  0.0003, -0.0492])
```

Zadania

Zadanie1

Załaduj polski model kontekstowych embeddingów Flair:

```
embeddings = FlairEmbeddings('pl-forward')
```

```
2026-10-15 11:15:16,884 https://flair.informatik.hu-berlin.de/resources/embeddings/flair/lm-polish-forward-v0.2.pt not found in cache, downloading to /t
100%|██████████| 80.3M/80.3M [00:04<00:00, 20.1MB/s]2026-10-15 11:15:21,454 copying /tmp/tmp0ssxyi0g to cache at /root/.flair/embeddings/lm-polish-forwa

2026-10-15 11:15:21,585 removing temp file /tmp/tmp0ssxyi0g
```

```
s1 = Sentence("Zamek w Malborku jest bardzo stary.")
s2 = Sentence("Suwak w kurtce to zamek błyskawiczny")
s3 = Sentence("Sprawdź, czy zamek w drzwiach działa")
sentences = [s1, s2, s3]
```

Oblicz embeddingi dla wszystkich zdań:

```
embeddings.embed(sentences)
```

```
[Sentence[7]: "Zamek w Malborku jest bardzo stary.",
 Sentence[6]: "Suwak w kurtce to zamek błyskawiczny",
 Sentence[7]: "Sprawdź, czy zamek w drzwiach działa"]
```

Wypisz każde zdanie i dla słowa "zamek", wypisz pierwsze 5 wymiarów embeddingu:

```
for sentence in sentences:
 for token in sentence:
  if token.text == "zamek":
     print(token.embedding[0:5])
```

```
tensor([-0.0043, -0.0131,  0.0004, -0.0001, -0.0004])
tensor([-4.3021e-03, -2.1082e-02,  7.7413e-04, -6.9931e-05, -3.6798e-04])
```

Czy embeddingi są różne w zależności od znaczenia słowa?

Odpowiedź: Tak

Zadanie 2

Stwórz bota z bazą przykładownych pytań i odpowiedzi, zwracającego odpowiedź na zapytanie na podstawie podobieństwa (WMD albo flair document embeddings) pomiędzy zapytniem użytownika a pytaniami w swojej bazie

Przykład:

Zbiór pytań i odpowiedzi :

```
[
  {"q1" : "a1"},
  {"q2" : "a2"},
  {"q3" : "a3"},
```

```
  ...
  {"qn" : "an"},
]
```

Wektoryzujemy pytania:

```
  {"q1_vecotorized" : "a1"},
  {"q2_vecotorized" : "a2"},
  {"q3_vecotorized" : "a3"},
  ...
  {"qn_vecotorized" : "an"},
```

Dla pytania użytkownia pytania q_user wektoryzujemy je i liczmy podobieństwo z pytaniami w bazie

```
sim("q_user_vectorized","q1_vecotorized"), sim("q_user_vectorized","q2_vecotorized"), ... ("q_user_vectorized""qn_vecotorized")
```

Zwracamy odpowiedź do pytania z bazy najbardziej podobnego do pytania użytkownika

$$argmax(sim(q_{user}vectorized, q_ivecotorized))$$

Do liczenia podobnieństma można użyć np https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.distance.cosine.html ale wtedy robimy argmin bo

$$cosine\ distance = 1 - cosine\ similarity$$

```
    qa = [
      {"q": "W którym roku odbyła się Bitwa pod Grunwaldem?", "a": "1410"},
      {"q": "W którym roku odbyła się Bitwa pod Verdun?", "a": "1916"},
      {"q": "Jaka jest stolica Francji?", "a": "Paryż"},
      {"q": "Jakie studio stworzyło Wiedźmina 3?", "a": "CD PROJEKT RED"},
      {"q": "Kiedy wybuchła II wojna światowa?", "a": "1 września 1939"}
    ]
```

```
    # importy
    from scipy.spatial.distance import cosine
    from flair.data import Sentence
    from flair.embeddings import TransformerDocumentEmbeddings, DocumentPoolEmbeddings

    sentence_embeddings = DocumentPoolEmbeddings([FlairEmbeddings('news-forward')])

    for question in qa:
        sentence = Sentence(question["q"])
        sentence_embeddings.embed(sentence)
```

```
    question["q_vectorized"] = sentence.get_embedding()

def calculate_similarity(vec1, vec2):
    return 1 - cosine(vec1, vec2)


def bot(user_question):
    user_sentence = Sentence(user_question)
    sentence_embeddings.embed(user_sentence)
    user_vector = user_sentence.get_embedding()

    similarities = []
    for question in qa:
        sim = calculate_similarity(user_vector, question["q_vectorized"])
        similarities.append(sim)

    most_similar_index = similarities.index(max(similarities))

    print(f"Pytanie użytkownika: {user_question}")
    print(f"Najbardziej podobne pytanie: {qa[most_similar_index]['q']}")
    print(f"Odpowiedź: {qa[most_similar_index]['a']}")
```

```
# TODO - zadaj 3 pytania
bot("Kiedy wybuchła II wojna światowa?")
bot("Jaka jest stolica Polski?")
bot("W którym roku odbyła się największa bitwa średniowieczna w Europie?")
```

```
Pytanie użytkownika: Kiedy wybuchła II wojna światowa?
Najbardziej podobne pytanie: Kiedy wybuchła II wojna światowa?
Odpowiedź: 1 września 1939
Pytanie użytkownika: Jaka jest stolica Polski?
Najbardziej podobne pytanie: Jaka jest stolica Francji?
Odpowiedź: Paryż
Pytanie użytkownika: W którym roku odbyła się największa bitwa średniowieczna w Europie?
Najbardziej podobne pytanie: W którym roku odbyła się Bitwa pod Grunwaldem?
Odpowiedź: 1410
```